

Minecraft: Multi-agent World Model for Continuous Learning Agents

31 MAY 2026 – AGI House Hackathon Final Report

Luke Hollis // lhollis@g.harvard.edu

Mindcraft

Multi-agent world model for continuous learning agents inside Minecraft.

GitHub repository Live demo: mindcraft.fyi

Continuous learning agents inside Minecraft based on Andrej Karpathy's *Animals v. Ghosts*.

Animals v. Ghosts

Karpathy argues that LLMs are more like ghosts than animals because their intelligence is distilled from existing human documents. He writes:

“We do not in fact have an actual, single, clean, actually bitter lesson pilled, ‘turn the crank’ algorithm that you could unleash upon the world and see it learn automatically from experience alone.”

Mindcraft is a hackathon-scale attempt at that direction: agents learn from embodied interaction in a world rather than only from static data.

What We Built

Minecraft is a runnable multi-agent Minecraft orchestration harness where generic agents learn from their own embodied experience.

- Each agent chooses skills, scores outcomes, stores replay, and updates persistent skill memory.
- The harness trains an action-conditioned world model across episodes.
- Agents improve by imagining their own Minecraft rollouts inside the continual-learning loop.
- The same continual-learning pattern is mirrored in Isaac Lab for Unitree Go2 Pro.

Live Demo

- **mindcraft.fyi:** live dashboard for Minecraft agents, snapshots, society state, learning graphs, and WebGL panes.
- **Minecraft society video:** `mindcraft_society.mp4` shows agents coordinating to mine, craft, trade, and advance inventory progression.
- **Go2 robotics video:** `go2_continual_learning_multiagent.mp4` shows the same continual-learning pattern in Isaac Lab.

The dashboard and videos are presentation surfaces. The proof artifacts are replay, checkpoints, W&B/Weave logs, TensorBoard, and eval reports.

Hackathon Criteria

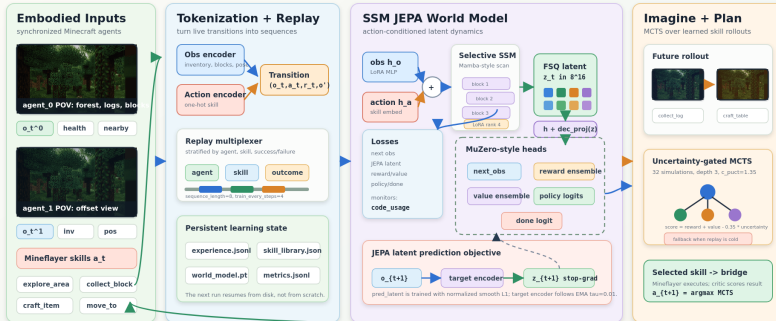
Criterion	Minecraft evidence
Agent orchestration	12 generic agents in one Minecraft world, coordinated through a Python orchestrator and Mineflayer bridge.
Utility	A reusable harness for online embodied learning, world modeling, skill memory, multi-agent resource sharing, and robotics transfer.
Technical execution	Dockerized Minecraft stack, WebGL dashboard, W&B/Weave, TensorBoard, resumable checkpoints, replay merge, distributed actor shards.
Creativity	Selective-Mamba-style world model with FSQ, JEPA latent prediction, MuZero heads, LoRA adapters, and MCTS skill imagination.
Sponsor usage	W&B tracks learning metrics and artifacts; Weave traces agent steps and orchestration calls.

Model Architecture

Minecraft Model Architecture

Minecraft experience -> typed replay -> selective SSM + JEPA/FSQ latents -> imagined MCTS skills -> persistent adaptation

Config anchors: seq=8, batch=16, model_dim=128
layers=3, latent_dim=16, FSQ bins=8, LoRA rank=4



Minecraft render in frames. Cinematic Craft screenshot from [Rothchild](#), CC-BY 4.0. Diagram is a project figure and is not an official Minecraft service or [Replay](#) Minecraft asset.

Architecture SVG asset: [figures/continual_learning_world_model_architecture.svg](#)

Hackathon Goals

Continual learning goals for Minecraft

A hackathon harness that turns live multi-agent experience into measurable skill progress.



Act in a real world

Mineflayer bots observe, move, mine, craft, trade, and fight in one shared Minecraft server.



Learn from replay

Every transition writes JSONL, updates skill memory, and trains the compact world model.



Plan with imagination

MCTS searches skill rollouts through reward, value, policy, and uncertainty heads.



Prove progress

Eval compares first vs last windows and logs proof metrics to W&B.

Latest measured proof window

PASS

4/4 behavioral checks

+33

stage advances

+5

agents at stage 5+

6

max stage reached

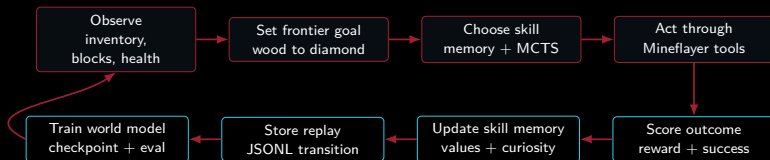
11,728

eval transitions

16,283

train step

Online Learning Loop



World Model

Part	Implementation
Backbone	Selective-Mamba 2 based SSM dynamics model for action-conditioned Minecraft sequences.
Latents	FSQ latent codes with JEPA-style latent prediction for compact next-state modeling.
Heads	MuZero-style reward, value, policy, done, unlock, and affordance prediction.
Adaptation	Optional LoRA adapters for efficient adaptation to new environments.
Planning	MCTS rolls candidate skills through the world model and penalizes uncertain predictions.
Replay	Persistent JSONL replay with per-agent sequence windows, validation holdouts, and progression-aware sampling.

Minecraft Orchestration

Layer	Current run
World	Paper Minecraft 1.21.1, Mineflayer bridge, WebGL dashboard.
Agents	12 generic agents in one shared world; 9 active WebGL panes for demo visibility.
Skills	Explore, recover, mine, craft, trade, fight, and progress through resource dependencies.
Mesh	trantor, prime, and pgx DGX Spark boxes connected as actor/replay shards.
Training	Combined replay trainer with resumable <code>world_model_mesh.pt</code> .
Telemetry	W&B/Weave online, TensorBoard on port 6006, dashboard on port 8790.

W&B And Weave

W&B tracks rewards, success rates, replay growth, inventory progress, skill values, and world-model losses. Weave traces agent steps and orchestration calls for multi-agent behavior.

Channel	Logged evidence
W&B trainer path	world_model/*, replay/*, and verified learning_eval/* proof metrics from train-replay.
W&B artifact run	Uploaded agent_space.svg, learning_curves.svg, Markdown report, and eval JSON.
Weave	Agent-step and orchestration-call trace path for multi-agent behavior.
TensorBoard	Local fallback for model losses, replay size, progress, and learning eval metrics.
JSONL	Full replay, training metrics, progress metrics, and learning eval metrics are reproducible offline.

Eval: Proof That Learning Happened

Metric	First 500	Last 500	Delta
Success rate	0.816	0.882	+0.066
Direct success rate	0.792	0.847	+0.055
Average reward	-0.455	0.252	+0.706
Max stage	4	6	+2
Stage advances	0	33	+33
Agents at stage 5+	0	5	+5

Eval verdict: **PASS, 4/4 behavioral checks**. Later replay windows contain new stage advances, higher reward, and later-stage capabilities absent in the first proof window.

Eval Graphs

Learning evidence curves

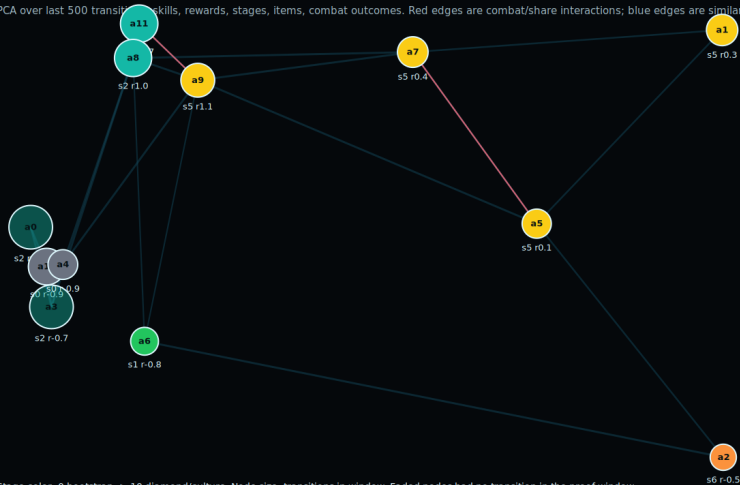
Each point is a chronological replay bin. Rising stage and milestone signals are behavioral progress; falling training losses are model fit.



Agent Space

Agent behavior space

PCA over last 500 transitions: skills, rewards, stages, items, combat outcomes. Red edges are combat/share interactions; blue edges are similar behavior.



Stage color: 0 bootstrap -> 10 diamond/culture. Node size: transitions in window. Faded nodes had no transition in the proof window.

Progression Evidence

Evidence	Observed in latest eval
Team item maxima	planks 4, sticks 28, crafting tables 13, wooden pickaxes 3, cobblestone 36, furnace 1, stone pickaxes 10, coal 2.
All-time milestones	coal 10, cobblestone 14, crafting tables 436, logs 1, planks 7, sticks 2, stone pickaxes 19, wooden pickaxes 7.
Agent stages	Five agents reached stage 5+; best agent reached stage 6, furnace-ready.
Combat	Duel behavior exists, with empty-kill rewards capped and loot-kill reward separated to avoid reward hacking.

Real-World Use

In the real world, robots such as Unitree Go2 Pro will need continuous learning from their environment using methods like the small Minecraft world model.

- We used Isaac Sim / Isaac Lab and the Go2 Pro robot to implement the continuous-learning model in a simulated environment.
- The Go2 demo mirrors the Minecraft loop: skill selection, replay, world-model training, and evaluation.
- Many robotic systems will need to coordinate the way Minecraft agents coordinate to trade, mine, craft, and complete shared tasks.

Robotics Demo

- **Video:** `go2_continual_learning_multiagent.mp4` shows the world model running on Go2 agents in Isaac Lab.
- **Module:** `mindcraft.robotics` contains Unitree Go2 replay, skill memory, and world-model training.
- **Script:** `scripts/isaac_go2_continual_demo.py` runs headless/vectorized Isaac Lab with optional video and cooperative goals.
- **Trace:** `go2_multiagent_orchestration.jsonl` records shared multi-agent skill-selection events.

What Makes It Continual

- Agents continue acting while replay grows.
- The model resumes from checkpoint instead of starting over.
- Skill values, preconditions, curiosity scores, and cooldowns persist.
- Replay is reused for dynamics, reward/value, policy, unlock, and affordance prediction.
- Eval compares early replay to recent replay and logs proof metrics.
- The same pattern transfers from Minecraft to simulated robotics.

Where It Is Still Honest

Minecraft is not yet the clean, fully bitter-lesson “turn the crank forever” algorithm.

- The skill vocabulary and some preconditions are still hand-designed.
- Rewards are shaped to make the hackathon horizon measurable.
- MCTS depends on a learned model that is still small and noisy.
- Iron and diamond remain frontier goals, not solved capabilities.

But the system is already beyond a scripted bot: later behavior depends on logged experience, replay training, persistent memory, model predictions, and measurable progression.

Demo Commands

```
mindcraft train-replay \  
  --storage-dir runs/default \  
  --follow --batches 0 \  
  --batch-size 16 \  
  --sequence-length 8 \  
  --telemetry \  
  --telemetry-run-name mindcraft-replay
```

```
PYTHONPATH=$PWD /home/trantor/Research/computational_robotics/IsaacLab/isaacclab.sh \  
  -p scripts/isaac_go2_continual_demo.py \  
  --headless \  
  --num_envs 4 \  
  --video \  
  --cooperative-goal
```

Final Takeaway

Mindcraft is a working, instrumented step toward animal-like continual learning: multiple agents act in a real world, learn from their own experience, train a compact world model, and produce auditable evidence of progress.

The hackathon win condition is not a perfect Minecraft player. It is a sophisticated, observable harness that can keep learning after the demo ends, and can transfer the same pattern toward robotics.